

INFORMATIQUE

2024 - 2025

Introduction à Java

Louis Sébastien SIMARD

SUP^F_C

Introduction à Java

Version 1

Table des matières

1. Introduction à Java	5
1.1 Introduction.....	5
1.2. Installation du JDK	5
1.2.1. Windows.....	5
2. Les bases du langage Java	9
2.1. Les types de base.....	10
2.2. Les classes String et StringBuffer.....	11
2.2.1. Exercices	13
2.3. Les opérateurs	14
2.3.1. Les opérateurs arithmétiques	14
2.3.2. Les opérateurs unaires	15
2.3.3. L'opérateur ternaire	15
2.4. La conversion de types (Transtypage).....	15
2.5. Les branchements conditionnels.....	16
2.5.1. if ... else.....	16
2.5.2. switch	17
2.6. Les boucles	17
2.6.1. while	18
2.6.2. for	19
2.7. Instructions d'interruption	19
2.7.1. break.....	19
2.7.2. continue.....	20
2.7.3. Les expressions rationnelles.....	20
2.7.4. Exercices	21
2.7.4.1. Exercice 1.....	21
2.7.4.2. Exercice 2.....	21
2.7.4.3. Les nombres pairs de 2 à 100	22
2.7.4.4. Test des limites.....	22
2.8. Les tableaux.....	23
2.8.1. Exercices	24
2.8.1.1. Tri à bulles	24
2.8.1.2. Création d'une matrice.....	24
2.8.1.3. Tri d'un tableau de chaînes de caractères.....	24

2.8.1.4. Extraction de mots	25
2.9. Le passage des arguments à une méthode	25
2.10. Les variables locales	26
2.11. Les packages	27
2.12. Contrôle d'accès en Java	28
2.13. Les exceptions	29
2.13.1. Les exceptions métiers	32
2.13.2. L'instruction finally	33
2.13.3. Exemple complet	33
2.14. Mot-clé this	34
3. Maven	34
3.1 Qu'est-ce que Maven et pourquoi faut-il l'utiliser ?	34
3.2 Mettre en place Maven	35
3.3 Créez un premier projet	36
3.4 Gérez les dépendances de votre projet	38
3.5 Un POM d'exemple complet	39
3.6 Profitez des fonctionnalités de Maven	40
4. Les fichiers JAR	43
5. Correction des Exercices	45

1. Introduction à Java

Cette série d'articles présente les éléments de base nécessaires pour commencer à programmer avec le langage Java. Cette section ne prétend pas se substituer à un cours complet sur le langage.

1.1 Introduction

Ce module ne fournit pas de cours exhaustif sur le langage Java : il en existe déjà de nombreux sur Internet. Pour obtenir un cours détaillé, nous vous conseillons le document « Développons en Java » disponible sur la page du cours.

Les besoins en outils pour réaliser les exercices ou étudier les exemples du cours sont peu nombreux. Nous aurons besoin :

- d'un éditeur de texte quelconque. Il est tout-à-fait possible d'utiliser Eclipse, mais cela n'est pas indispensable,
- du kit de développement JDK 8 disponible sur le site officiel www.oracle.com/technetwork/java. Pour les nouveaux utilisateurs d'Eclipse, voici un document PDF permettant d'appréhender cet outil : [Développons en Java avec Eclipse](#).

1.2. Installation du JDK

En fonction de l'OS, l'installation du JDK peut être différente. Nous allons rapidement présenter l'installation du JDK sur le système Windows.

1.2.1. Windows

La version du JDK dans l'exemple est la 8 update 121. La version que vous allez utiliser sera peut-être différente. Il vous faudra certainement adapter les chemins.

Le JDK est accessible sur le site officiel de Java

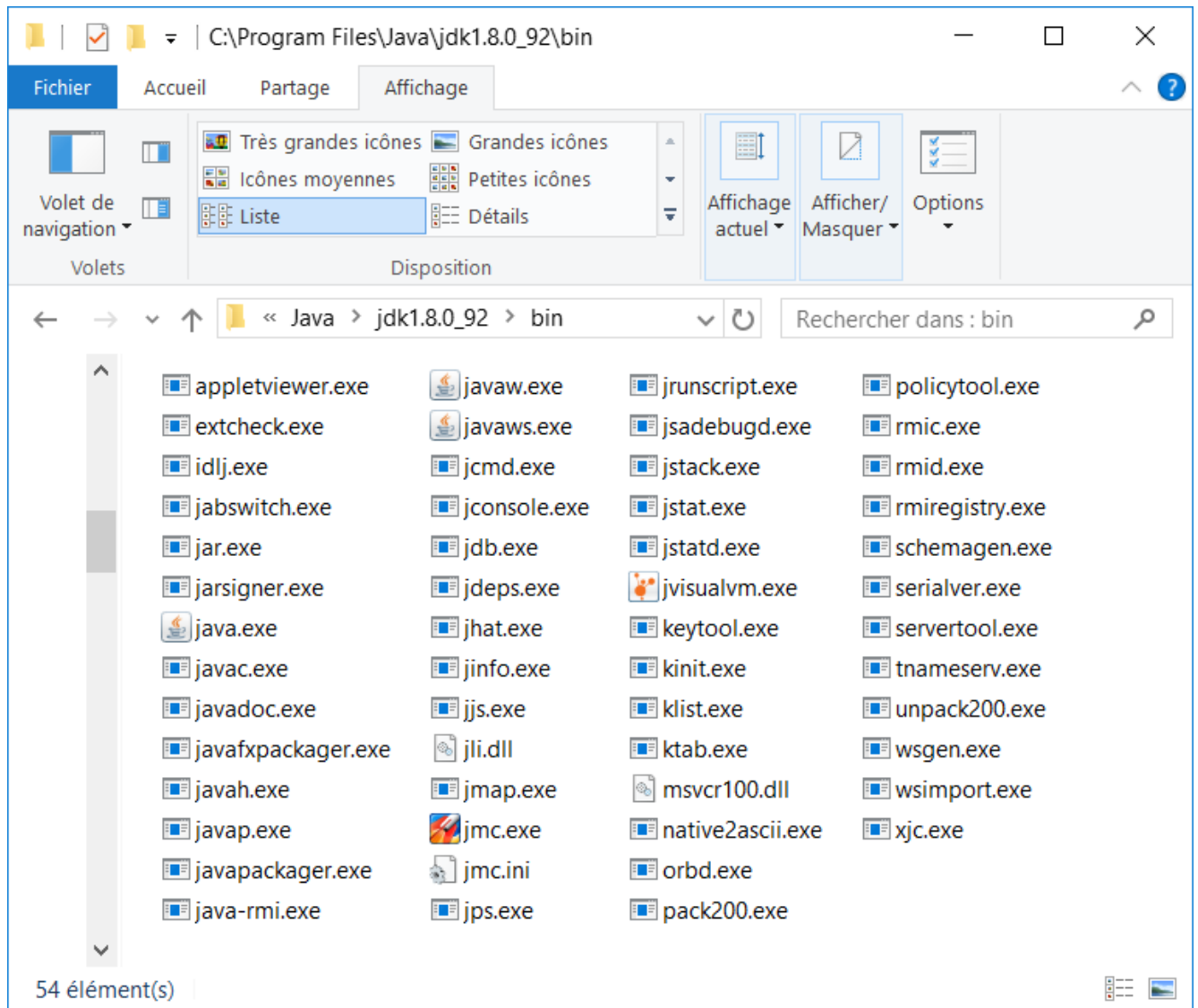
<http://www.oracle.com/technetwork/java/javase/downloads/index.html>¹.

Pour pouvoir compiler il est important de télécharger le JDK et non la JRE uniquement.

Nous lançons le programme d'installation et nous ne modifions aucun des paramètres proposés.

L'installation terminée, nous recherchons à quel emplacement le JDK a été installé. Il doit être situé dans le dossier C:\Program Files\Java\jdk1.8.0_92. Voici une copie d'écran pour ma machine :

¹ Le contenu du site peut subir des modifications concernant son contenu.



Dossier d'installation du JDK sous Windows

Les outils² du JDK ont été installés dans un dossier nommé C:\Program Files\Java\jdk1.8.0_92\bin.

Pour contrôler que le JDK c'est bien installé il faut ouvrir une console texte s'électionnant l'application « Invite de commandes » du menu « Démarrer ».

Entrer la commande javac qui correspond au compilateur. La commande doit être trouvée et produire un résultat comme celui-ci :

² Compilateur, générateur de documentation et d'archives, etc.

```

Invite de commandes

C:\Users\louis>javac
Usage: javac <options> <source files>
where possible options include:
  -g                  Generate all debugging info
  -g:none             Generate no debugging info
  -g:{lines,vars,source}  Generate only some debugging info
  -nowarn             Generate no warnings
  -verbose            Output messages about what the compiler is doing
  -deprecation        Output source locations where deprecated APIs are used
  -classpath <path>   Specify where to find user class files and annotation processors
  -cp <path>          Specify where to find user class files and annotation processors
  -sourcepath <path>  Specify where to find input source files
  -bootclasspath <path> Override location of bootstrap class files
  -extdirs <dirs>     Override location of installed extensions
  -endorseddirs <dirs> Override location of endorsed standards path
  -proc:{none,only}   Control whether annotation processing and/or compilation is done.
  -processor <class1>[,<class2>,<class3>...] Names of the annotation processors to run; bypasses default discovery process
  -processorpath <path> Specify where to find annotation processors
  -parameters         Generate metadata for reflection on method parameters
  -d <directory>      Specify where to place generated class files
  -s <directory>      Specify where to place generated source files
  -h <directory>      Specify where to place generated native header files
  -implicit:{none,class} Specify whether or not to generate class files for implicitly referenced files
  -encoding <encoding> Specify character encoding used by source files
  -source <release>   Provide source compatibility with specified release
  -target <release>   Generate class files for specific VM version
  -profile <profile>  Check that API used is available in the specified profile
  -version            Version information
  -help              Print a synopsis of standard options
  -Akey[=value]       Options to pass to annotation processors
  -X                 Print a synopsis of nonstandard options
  -J<flag>            Pass <flag> directly to the runtime system
  -Werror             Terminate compilation if warnings occur
  @<filename>         Read options and filenames from file

C:\Users\louis>

```

Test appel du compilateur javac depuis une console sous Windows

Le compilateur est maintenant disponible de partout. Nous pouvons éditer un petit programme Java dans un dossier que nous aurons créé au préalable, par exemple le dossier C:\Alao. Pour éditer notre programme, nous pouvons utiliser l'éditeur SCite. Cet éditeur complet permet d'écrire, de compiler et de tester nos programmes.

The screenshot shows the SciTE editor window titled "HelloWorld.java - SciTE". The menu bar includes File, Edit, Search, View, Tools, Options, Language, Buffers, and Help. The file list shows "1 HelloWorld.java". The editor contains the following Java code:

```

public class HelloWorld {
    public static void main( String[] args) {
        System.out.println( "Hello world !");
    }
}

```

On the right side of the editor, the output of the compilation and execution is shown:

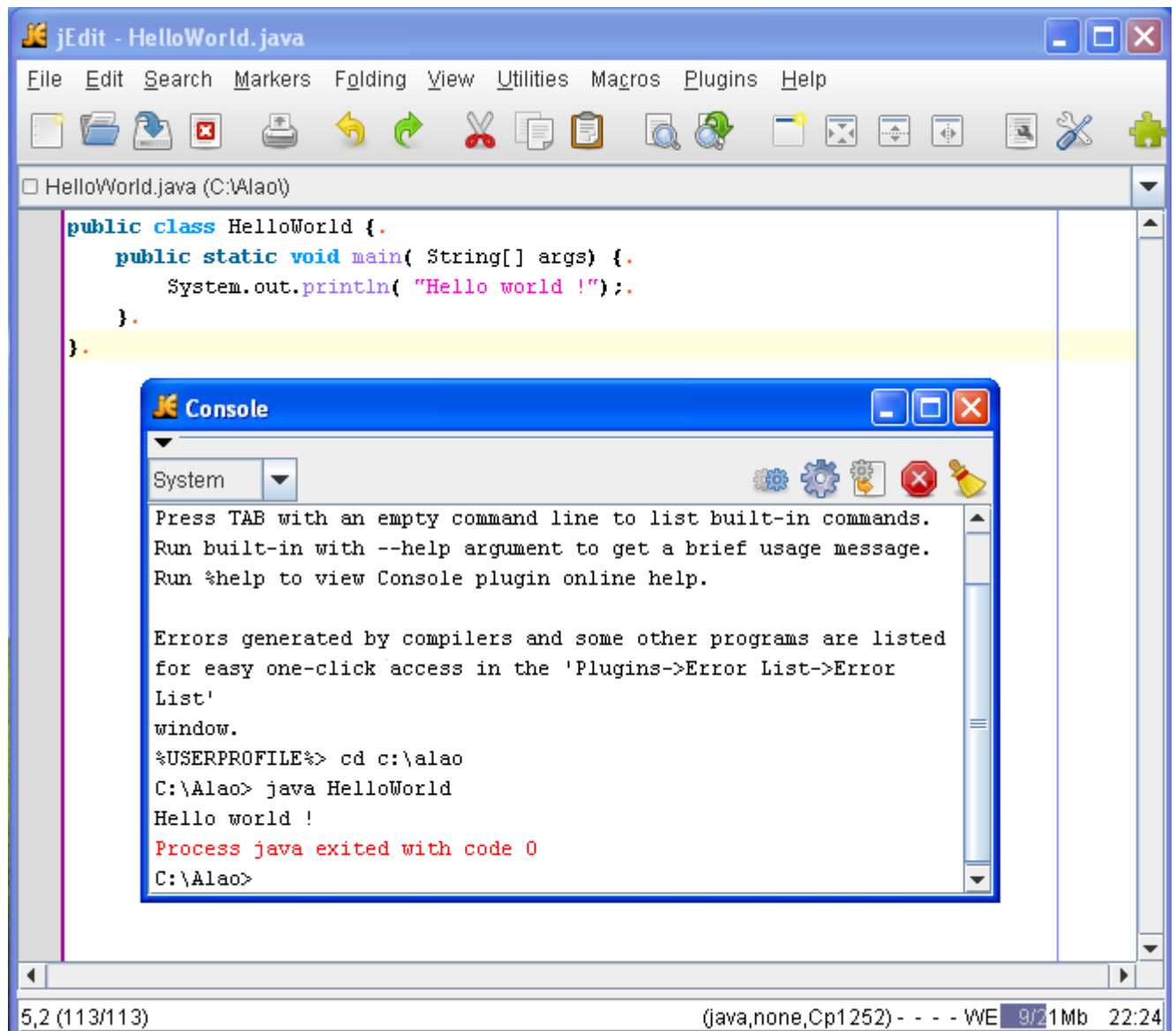
```

>javac HelloWorld.java
>Exit code: 0
>java -cp . HelloWorld
Hello world !
>Exit code: 0

```

Editeur SCite sous Windows

Nous avons aussi JEdit, éditeur de texte écrit en Java, très agréable à utiliser. Il possède de nombreux plugins qui permettent d'étendre ses possibilités.



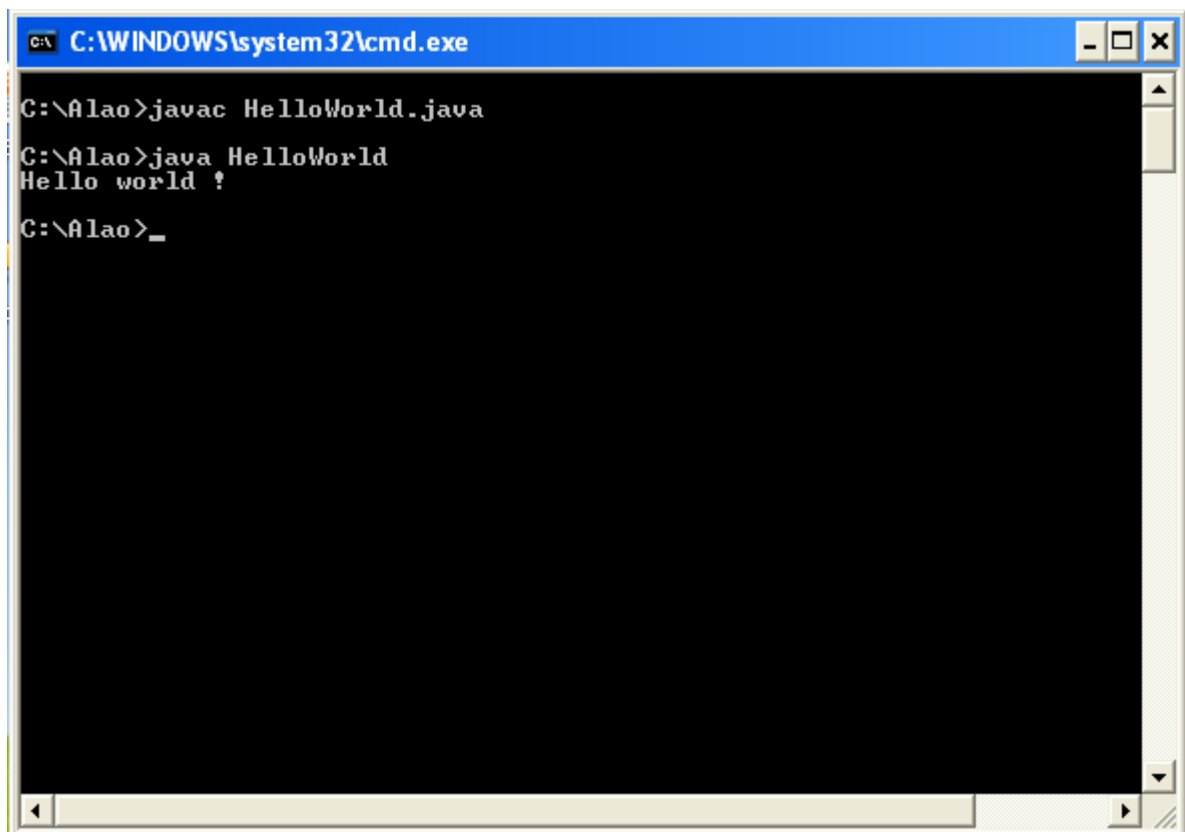
Editeur JEdit sous Windows

SCite et JEdit

Les éditeurs SCite et JEdit sont des logiciels disponibles gratuitement sur Internet. Les moteurs de recherche vous donneront sans problème des liens pour les télécharger afin de les installer.

La compilation en ligne de commande se fait en utilisant le compilateur javac. Si une erreur de syntaxe est présente dans notre code, le compilateur nous en informe et nous devons la corriger. Quand la compilation réussit, nous pouvons lancer le programme avec la commande java. La compilation produit un fichier ayant l'extension « class ». Ce fichier contient le pseudo-code qui est utilisé par la JVM (Java Virtual

Machine). Nous pouvons observer que nous passons au programme java l'argument HelloWorld sans l'extension class. Le texte « Hello world ! » est alors imprimé à l'écran.



```
C:\WINDOWS\system32\cmd.exe

C:\Alao>javac HelloWorld.java
C:\Alao>java HelloWorld
Hello world !
C:\Alao>_
```

Compilation et exécution d'un programme sous Windows

2. Les bases du langage Java

Dans ce chapitre, nous allons passer en revue les éléments fondamentaux de langage. Les concepts objets du langage seront étudiés dans le prochain chapitre. Ces éléments sont faciles à appréhender car très similaires à ceux que l'on peut rencontrer dans d'autres langages, en particulier le langage C. Pour notre bonheur, Java a, de plus, simplifié et sécurisé beaucoup de concepts. Avant de nous lancer dans cette étude, nous allons présenter le programme Java minimal : le fameux « Hello world » :

1. Exemple 2.1. Le programme « Hello world »

```
// HelloWorld.java
class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello world");
    }
}
```

Le code de ce programme doit être situé dans un fichier portant le nom *HelloWorld.java*. La compilation de ce programme est réalisée avec le compilateur *javac* :

```
# javac HelloWorld.java
```

Le résultat de la compilation est le fichier *HelloWorld.class*. Pour lancer l'exécution du pseudo-code, nous utilisons la JVM accessible via le programme *java*. Notez que nous passons en paramètre la chaîne « HelloWorld » sans le « .class » :

```
# java HelloWorld
Hello world
```

Si vous voyez la chaîne « Hello world » s'afficher sans erreur, alors votre environnement de développement fonctionne parfaitement. Dans le cas contraire, décrivez les erreurs rencontrées sur le forum du module ALAO, afin que nous puissions résoudre les problèmes.

2.1. Les types de base

Malgré son orientation vers du tout objet, Java a conservé l'utilisation des types de base. La raison est de conserver des performances raisonnables selon le contexte. Les types de base sont similaires à ceux que l'on trouve en C/C++. Néanmoins, il existe des différences notables par rapport à ces langages très proches de l'architecture du processeur :

- Le type char est toujours codé sur 16 bits pour être conforme à l'Unicode.
- Le type entier int est toujours signé.
- Quel que soit la plateforme (32 ou 64 bits), les entiers sont toujours codés sur 32 bits.
- Le type boolean est un type à part entière. Il est codé sur 1 bit.

2. Tableau 2.1. Les types de base

Type de donnée	Taille en bits	Description
byte	8	Entier signé
short	16	Entier signé
int	32	Entier signé
long	64	Entier signé
float	32	Réel
double	64	Réel
char	16	Réel
booléen	1 bit	true ou false

L'utilisation d'un type de base nécessite de déclarer une variable. La déclaration d'une variable respecte la syntaxe suivante :

```
type_donnée nom_variable;
```

Il est possible d'affecter une valeur à la variable lors de la déclaration.

3. Exemple 2.2. Déclaration de types de base

```
int entier1;  
int entier2 = 50;  
float reell;  
char caractere = 'a';  
boolean test = true;
```

Remarque

Comme en C/C++, une variable n'existe que dans le bloc où elle a été déclarée. A la sortie du bloc, par exemple une fonction, cette variable sort de la portée : elle est donc détruite.

Il n'existe pas de type de base « chaîne de caractères » en Java. Le type *String* est une classe d'objet. Java offre toutefois des facilités de syntaxe qui peuvent nous le faire considérer comme un type de base. En particulier, nous pouvons déclarer un objet *String* sans utiliser l'instruction *new*, alors que c'est obligatoire pour tous les autres types d'objets.

4. Exemple 2.3. Déclaration d'une variable de type String

```
String chaine1 = "Bonjour";  
// Identique à la ligne précédente  
String chaine2 = new String("Bonjour");
```

5. Exemple 2.4. TypeBase.java

```
// TypeBase.java  
public class TypeBase {  
    public static void main(String[] args) {  
        char car1, car2 = 'A', car3 = 97;  
        String chaine1 = "Un texte";  
        long long1 = 10L;  
  
        System.out.println( "Liste de variables declarees : " );  
        System.out.println( "car2      : " + car2 + "\n" +  
                             "car3      : " + car3 + "\n" +  
                             "chaine1   : " + chaine1 + "\n" +  
                             "long1     : " + long1 );  
    }  
}
```

2.2. Les classes String et StringBuffer

Nous avons vu les types de base, ou types primitifs fournis par le langage Java. Les chaînes de caractères n'existent pas en Java et, contrairement aux langages C et C++, les tableaux de caractères ne sont assimilables qu'à des chaînes de caractères.

Bien que les classes *String* et *StringBuffer* ne soient pas des types de bases, il est difficile de travailler en Java sans les aborder.

La classe *String* permet de représenter une chaîne de caractères sous la forme d'un objet. Cette classe propose plusieurs constructeurs. Nous pouvons en obtenir la liste en consultant la documentation. Elle fournit également plusieurs méthodes permettant de manipuler la chaîne de caractères qu'elle encapsule. Soit l'objet *str* construit comme suit :

```
String str = "Java est un langage objet";
```

- *str.toString()* : retourne la chaîne de caractères associée à l'instance, soit "Java est un langage objet".
- *str.indexOf('u')* : retourne la position du caractère 'u' soit la valeur 9. Le premier caractère est à la position 0.
- *str.substring(0,4)* : retourne la sous-chaîne en partant de la position 0 jusqu'à la position 4 exclue. Nous obtenons donc la sous-chaîne "Java".
- *str.charAt(2)* : retourne le caractère 'v'.
- *str.toUpperCase()* : retourne une chaîne de caractères en majuscules, soit "JAVA EST UN LANGAGE OBJET".

Il existe plus d'une cinquantaine de méthodes ! Seules quelques-unes sont régulièrement utilisées.

Nous pouvons concaténer des chaînes de caractères avec l'opérateur *+*. La méthode *concat* permet de réaliser la même chose.

```
String str = "Java est un langage objet";  
str = str + ". Java est un langage portable";
```

En Java, une chaîne de caractères est immuable. Elle ne peut donc pas être modifiée. Les méthodes qui retournent des chaînes de caractères retournent un nouvel objet. Cela peut dégrader considérablement les performances d'un programme dans certaines situations. Dans l'exemple précédent, après l'affectation, *obj* est un nouvel objet. Pour résoudre ce problème, Java fournit la classe *StringBuffer*.

La classe *StringBuffer* permet de manipuler le contenu d'une chaîne de caractères sans la nécessité de créer un nouvel objet à chaque modification. Si un traitement sur une chaîne de caractères impose de nombreuses modifications sur celle-ci, il faut alors éviter d'utiliser la classe *String* et préférer la classe *StringBuffer*.

StringBuffer propose entre autres les méthodes *append* et *insert*.

```
StringBuffer obj = "Java est un langage objet";  
obj.append(". Java est un langage portable");
```

6. Exemple 2.5. Comparaison entre *String* et *StringBuffer*

```
public class TestStringBuffer {  
    public static void main(String[] args) {  
        if ( args.length == 1 && args[0].equals("good") ) {  
            System.out.println("sayGoodHello");  
            sayGoodHello();  
        }  
        else {  
            System.out.println("sayBadHello");  
        }  
    }  
}
```

```

        sayBadHello();
    }
}

public static String sayBadHello() {
    String bonjour = "bonjour";

    for(int i=1; i < 100000; ++i) {
        bonjour += i;
    }
    return bonjour;
}

public static String sayGoodHello() {
    StringBuffer bonjour = new StringBuffer(20);
    bonjour.append("bonjour");

    for(int i=1; i < 100000; ++i) {
        bonjour.append(i);
    }
    return bonjour.toString();
}
}

```

L'exécution du programme donne les résultats suivants :

```

~$ time java TestStringBuffer good
sayGoodHello

real    0m0.191s
user    0m0.104s
sys     0m0.028s
~$
~$ time java TestStringBuffer
sayBadHello

real    1m13.184s
user    1m12.569s
sys     0m0.240s
jmcaricand@amande:~$

```

Les résultats ne peuvent pas être contestés. Lorsqu'il s'agit de modifier la même chaîne de caractères lors d'un traitement, nous devons utiliser la classe *StringBuffer*.

2.2.1. Exercices

Ecrire une méthode qui prend comme argument une chaîne de caractères délimitée pour stocker un nom et une adresse. Cette méthode doit transformer cette chaîne et retourner une chaîne qui pourrait être imprimée sur une étiquette.

Par exemple, si la chaîne contient le texte suivant :

Jean-Michel Caricand:16 route de Gray:25030:Besançon

la méthode doit retourner le texte suivant :

Comme vous pouvez le constater, le nom de la personne est transformé en majuscules.

[Correction en annexe.](#)

2.3. Les opérateurs

Tout comme les types de base, les opérateurs sont semblables à ceux de C/C++. Nous ne donnerons que quelques exemples afin d'illustrer certains opérateurs.

7. Tableau 2.2. Les opérateurs

Arithmétiques	+, -, *, /, %
Affectation	=, +=, -=, *=, /=
Unaires	--, ++
Logiques bits à bits	~, &, ^, <<, >>
Comparaisons logiques	==, !=, <, <=, >, >=
Logiques sur les booléens	!, &&,
Ternaires	?:

2.3.1. Les opérateurs arithmétiques

Les opérateurs arithmétiques permettent d'effectuer des calculs sur les entiers et les réels. L'opérateur + est un opérateur valide pour le type *String*. Il n'additionne pas réellement les chaînes, mais réalise une concaténation.

8. Exemple 2.6. Opérateurs arithmétiques

```
// OpArithmetiques.java
public class OpArithmetiques {
    public static void main(String[] args) {
        int r1 = 10 + 20;
        int r2 = (int) 30L + 1;
        short i = 5000;
        short j = 4;
        int k = i + j;
        String c = "Un texte";

        System.out.println( "r1          : " + r1  + "\n" +
                           "r2          : " + r2  + "\n" +
                           "c + \" azerty\" : " + c + " + azerty" + "\n" +
                           "k          : " + k  );
    }
}
```

```
}  
}
```

2.3.2. Les opérateurs unaires

Ils permettent d'incrémenter ou de décrémenter une variable. Ils peuvent être préfixés ou postfixés. La différence est subtile :

9. Exemple 2.7. Opérateurs unaires

```
// OpUnaires.java  
public class OpUnaires {  
    public static void main(String[] args) {  
        int a = 0;  
        int b = 0;  
  
        System.out.println("Valeurs de départ de a et de b : " + a + ", " +  
b);  
  
        System.out.println("Valeur de ++a : " + ++a);  
        System.out.println("Valeur de a : " + a);  
  
        System.out.println("Valeur de b++ : " + b++);  
        System.out.println("Valeur de b : " + b);  
    }  
}
```

2.3.3. L'opérateur ternaire

Cet opérateur est une facilité d'écriture. Il peut facilement être remplacé par un if...else. Cet opérateur est très souvent utilisé par les développeurs C/C++. L'avantage de cet opérateur est essentiellement la concision du code.

La syntaxe est la suivante :

```
expression_testée ? valeur_si_vrai : valeur_si_faux;
```

10. Exemple 2.8. Opérateur ternaire

```
// OpTernaire.java  
public class OpTernaire {  
    public static void main(String[] args) {  
        int a = 100;  
        int b = 10;  
  
        int c = ( a > b ) ? a : b;  
  
        System.out.println("Valeur de c : " + c);  
    }  
}
```

2.4. La conversion de types (Transtypage)

Le transtypage permet de convertir un type de données en un autre type. Ce phénomène peut se produire implicitement lors d'une affectation. Par exemple :

```
short s;  
char c = 10;
```

Une variable de type *long* est codée sur 64 bits, une variable de type *int*, sur 32 bits. Lors de l'affectation de *i* à *l*, nous ne perdons aucune information, car un *long* peut contenir un *int*.

```
c = 00000000000001010  
s = 000000000000000000000000000001010
```

Le transtypage explicite peut faire perdre de l'information. Il est donc à manipuler avec précaution :

Exemple 2.9. Transtypage

```
// TranTypage.java  
public class TranTypage {  
    public static void main(String[] args) {  
        int i1 = 10, i2;  
        long l1, l2 = 1234567890123456789L;  
  
        l1 = i1;  
        i2 = (int) l2;  
        System.out.println("i1 : " + i1 + ", l1 : " + l1);  
        System.out.println("i2 : " + i2 + ", l2 : " + l2);  
    }  
}
```

La sortie de ce programme met en évidence le problème :

```
i1 : 10, l1 : 10  
i2 : 2112454933, l2 : 1234567890123456789
```

2.5. Les branchements conditionnels

2.5.1. if ... else

Cette structure conditionnelle permet d'exécuter ou non certaines portions de code. La syntaxe générale :

```
if ( expression booléene )  
    bloc d'instructions si vrai  
else  
    bloc d'instructions si faux
```

Plusieurs « if » différents peuvent être utilisés dans la même structure :

```
if ( expression booléene )  
    bloc d'instructions si vrai
```

```
else if ( expression booléene )
    bloc d'instructions si vrai
else
    bloc d'instructions si faux
```

Exemple 2.10. IfElse.java

```
// IfElse.java
public class IfElse {
    public static void main(String[] args) {
        int a = 100, b = 10, c =20;

        if ( c > a ) {
            // Bloc d'instruction. Il faut des accolades
            System.out.print("c est > ");
            System.out.println("à a");
        }
        else if ( c >= b )
            // Une seule instruction. Les accolades ne sont
            // pas nécessaires.
            System.out.println("c est >= b ");
        else
            System.out.println("c est strictement < à a et b");
    }
}
```

2.5.2. switch

L'instruction *switch* permet de réaliser des branchements conditionnels multiples. Elle peut être remplacée par l'instruction *if...else*. Cette instruction souffre d'une limitation importante : elle ne peut effectuer que des comparaisons sur des entiers ou assimilés. Elle répond toutefois à certains tests simples successifs. La syntaxe générale est :

```
switch ( valeur entière ) {
    case valeur_1:
        liste d'instructions;
        break;
    case valeur_2:
        liste d'instructions;
        break;
    default:
        liste d'instructions;
}
```

L'instruction *break* est importante dans chaque *case*. Elle permet de sortir de l'instruction *switch* sans effectuer les instructions appartenant aux autres *case*. *break* est une instruction d'interruption.

2.6. Les boucles

Les boucles permettent de répéter une instruction ou un bloc d'instructions tant qu'une expression booléenne est vraie. Il est important que la condition puisse être évaluée à faux à un certain moment pour éviter de boucler indéfiniment.

2.6.1. while

Java propose deux types de boucles *while* : *while* et *do while*.

La syntaxe de la boucle *while* est :

```
while ( expression booléenne vraie )  
    instruction; // ou bloc d'instructions
```

De par sa structure, la boucle *while* peut ne jamais être exécutée.

11. Exemple 2.11. BoucleWhile.java

```
// BoucleWhile.java  
public class BoucleWhile {  
    public static void main(String[] args) {  
        int compteur = 100;  
  
        while ( compteur > 0 ) {  
            System.out.println( compteur );  
            // compteur sera négatif à un moment  
            compteur--;  
        }  
  
        // Il faut prévoir une condition de sortie  
        // dans le corps de la boucle  
        while ( true ) {  
            if ( compteur > 50 ) break;  
            System.out.println( compteur );  
            compteur++;  
        }  
  
        // Ne sera jamais exécutée  
        while ( compteur < 50 )  
            System.out.println( "Valeur de compteur : " + compteur );  
    }  
}
```

La boucle *do while* est toujours exécutée une fois, car l'expression est évaluée à la fin du bloc d'instructions. Ce comportement est utile dans certains cas. La syntaxe est :

```
do  
    instruction; // ou bloc d'instructions  
while ( expression booléenne vraie );
```

12. Exemple 2.12. BoucleDoWhile.java

```
// BoucleDoWhile.java  
public class BoucleDoWhile {  
    public static void main(String[] args) {  
        int compteur = 0;  
  
        do {  
            System.out.println( ++compteur );  
        } while ( compteur < 5 );  
    }  
}
```

```
}  
}
```

2.6.2. for

La boucle *for* est la plus complète des boucles. Elle peut remplacer toutes les autres boucles, mais au prix d'une lourdeur du code. Elle est en tous points semblable à celle des langages C/C++. La syntaxe générale de la boucle *for* est :

```
for ( initialisation de variables;  
      expression booléenne;  
      instructions récurrentes )  
    instruction; // ou bloc d'instructions
```

Si la variable présente dans l'expression booléenne est déclarée dans la partie initialisation, alors la portée de cette variable est le bloc d'instruction de la boucle *for*. Exemple :

13. Exemple 2.13. BoucleFor.java

```
// BoucleFor.java  
public class BoucleFor {  
    public static void main(String[] args) {  
        for ( int compteur = 0; compteur <= 10; compteur++ )  
            System.out.println( compteur );  
  
        //System.out.println( compteur ); // Erreur de compilation  
    }  
}
```

2.7. Instructions d'interruption

2.7.1. break

L'instruction *break* permet d'interrompre l'exécution d'une boucle. Cela peut être utile dans certaines circonstances. *break* fait sortir immédiatement de la boucle en cours. Exemple :

14. Exemple 2.14. BoucleForAvecBreak.java

```
// BoucleForAvecBreak.java  
public class BoucleForAvecBreak {  
    public static void main(String[] args) {  
        for ( int compteur = 0; compteur <= 10; compteur++ ) {  
            System.out.println( compteur );  
            if ( compteur == 2 ) break;  
        }  
    }  
}
```

2.7.2. continue

L'instruction *continue* permet d'interrompre l'exécution du bloc d'instructions pour revenir immédiatement au test. Exemple :

15. Exemple 2.15. BoucleForAvecContinue.java

```
// BoucleForAvecContinue.java
public class BoucleForAvecContinue {
    public static void main(String[] args) {
        for ( int compteur = 0; compteur <= 10; compteur++ ) {
            if ( compteur == 2 ) continue;
            System.out.println( compteur );
        }
    }
}
```

2.7.3. Les expressions rationnelles

La package *java.util.regex* offre des outils qui permettent de décomposer une chaîne de caractères en morceaux selon un pattern défini.

Comme beaucoup de langages, la syntaxe des patterns est celle du langage Perl, langage qui excelle dans ce domaine.

Le package *java.util.regex* utilise les classes :

- *Pattern* : cette classe représente l'expression rationnelle à compiler,
- *Matcher* : cette classe représente le moteur de l'analyseur d'expression,
- *PatternSyntaxError* : cette classe est le type d'exception lancée si une erreur survient lors de l'analyse.

Un objet *Pattern* est créé à partir de l'une des deux méthodes statiques suivantes de classe *Pattern* :

- *static Pattern compile(String expression);*
- *static Pattern compile(String expression, int flags);*

16. Exemple 2.16. Utilisation des Regex

```
package Pattern;
import java.util.regex.*;

public class Demo {
    public static void main(String[] args) {
        String chaine = "A10B";
        String mots = "mot1:mot2:mot3";

        Pattern p1 = Pattern.compile("\\d+");
        Pattern p2 = Pattern.compile(":");

        Matcher m = p1.matcher(chaine);
        boolean b = m.find();
    }
}
```

```

    if (b) {
        System.out.println("La chaine : \"" + chaine + "\"" contient un
nombre");
    }

    String[] tableau = p2.split(mots);
    for (String s: tableau) {
        System.out.println(s);
    }
}

```

L'exemple précédent montre comment utiliser les regex. Il est également possible d'utiliser la notion de groupe avec la méthode *matches* de la classe *Matcher*. Cette fonction est plus complexe, mais très puissante. Elle dépasse le cadre de ce cours.

2.7.4. Exercices

2.7.4.1. Exercice 1

Ecrivez un programme Java qui accomplit les tâches suivantes :

- Somme des entiers impairs situés entre 1 et 99 en utilisant la boucle *for*.
- Calcul de la valeur de 2.5 à la puissance 3 en utilisant la méthode *pow*.
- Imprime les entiers de 1 à 20 en utilisant une boucle *while*. Utilisez la variable « i » dans l'expression booléenne de l'instruction *while*. Imprimez seulement cinq entiers par ligne³.

[Corrigé.](#)

2.7.4.2. Exercice 2

Trouvez les erreurs dans les blocs de code qui suivent, et expliquez comment les corriger :

```

i = 1;

while ( i <= 10 );
    i++;
}

for ( k = 0.1; k <= 1.0 ; k += 0.1 )
    System.out.println( k );

switch ( n )
{
    case 1:
        System.out.println( "Le nombre est 1" );
    case 2:
        System.out.println( "Le nombre est 2" );
        break;
    default:
        System.out.println( "Le nombre est différent de 1 et de 2" );
}

```

³ Utilisez la valeur du calcul *i%5*. Lorsque la valeur de cette expression vaut 0, imprimez un caractère de « retour chariot », sinon, imprimez un caractère *TAB*.

```
        break;
    }
```

Le code suivant devrait imprimer les valeurs 1 à 10 :

```
n = 1 ;

while ( n <> 10 )
    System.out.println( n++ );
```

[Corrigé.](#)

2.7.4.3. Les nombres pairs de 2 à 100

Trouvez les erreurs dans les blocs de code qui suivent, et expliquez comment les corriger :

Le code suivant devrait imprimer les nombres pairs compris entre 2 et 100 :

```
counter = 2;
do {
    System.out.println( counter );
    counter += 2 ;
} While ( counter < 100 );
```

[Corrigé.](#)

2.7.4.4. Test des limites

Ecrivez un programme qui définit et initialise une variable « i » de type *byte*. Cette variable est initialisée à la valeur 1. Multipliez cette variable par 2 huit fois de suite en affichant la valeur à chaque passage. Expliquez le résultat.

[Corrigé.](#)

2.7.4.5. Lettres aléatoires

Ecrivez un programme qui crée une séquence aléatoire de lettres majuscules. Vous ne devez conserver que les consonnes, donc éliminer les voyelles qui auraient pu être générées.

[Corrigé.](#)

2.7.4.6. Volume d'une sphère

Ecrivez un programme complet en Java qui demande à l'utilisateur d'entrer le rayon d'une sphère et appelle une méthode *calculerVolumeSphere* pour calculer le volume de la sphère. La formule de calcul est la suivante :

```
double volume = ( 4.0 / 3.0 ) * Math.PI * Math.pow( radius, 3 )
```

[Corrigé.](#)



Saisir des valeurs au clavier

La saisie d'une valeur au clavier est complexe sans utilitaire en Java. Une classe dédiée à la saisie de valeurs au clavier est disponible dans ce cours : [LectureClavier.java](#).

2.8. Les tableaux

Le langage Java offre dans le framework associé un ensemble de classes permettant de gérer des collections. Ces classes sont très puissantes et agréables à utiliser. Néanmoins, la notion de tableau de base est toujours présente. Le tableau fonctionne comme attendu. Nous devons le déclarer en indiquant le type d'objet qu'il peut contenir. A la construction du tableau, nous devons spécifier le nombre d'éléments qu'il pourra contenir. La taille d'un tableau ne peut pas être modifiée. Nous pouvons accéder aux éléments à l'aide des crochets.

Le tableau en Java est un objet à part entière. Déclarer un tableau ne suffit pas. Nous devons le construire à l'aide de l'instruction *new*.

```
int [] tableauEntiers;           // juste déclaré, pas encore construit !
tableauEntiers = new int[10];    // maintenant, le tableau est utilisable
```

Si nous connaissons déjà le contenu du tableau, nous pouvons utiliser la syntaxe suivante :

```
int [] tableauEntiers = { 1, 2, 3, 4, 5 };
```

Nous n'avons pas à donner la taille du tableau, car le compilateur peut la déduire de la liste d'initialisation.

Pour accéder à un élément, nous utilisons son index. Comme dans beaucoup de langages, le premier élément est à l'index 0.

```
int [] tableauEntiers = new int[10];
tableauEntiers[0] = 1;
System.out.println( tableauEntiers[0] );
```

Contrairement au langage C++, Java contrôle les débordements d'index. L'accès à un index inexistant lancera une exception qui pourra être traitée par le développeur.

A la création du tableau, les éléments sont automatiquement initialisés. Les nombres reçoivent la valeur 0, les booléens, la valeur *false* et les objets, la valeur *null*.

```
String [] tableauChaines = new String[5];           // Chaque élément contient la
valeur null.
tableauChaines[0] = new String("Elément 1");       // Maintenant, la case 0 contient
une référence                                     // à une String.
System.out.println( tableauChaines[0] );
```

Un tableau Java possède une propriété nommée *length* qui retourne la longueur de tableau.

```
String [] tableauChaines = new String[5];
tableauChaines[0] = new String("Elément 1");
System.out.println( "La longueur du tableau est " + tableauChaines.length );
```

Un tableau peut avoir plusieurs dimensions. Nous avons en réalité un tableau contenant des tableaux. Comme dans tous les langages, la création de tableaux multidimensionnels est inconfortable. Par exemple, pour créer un tableau d'entiers à 2 dimensions :

```
int [][] tableau;  
tableau = new int[2][];  
tableau[0] = new int[5];  
tableau[1] = new int[5];
```

Si nous connaissons les valeurs à affecter aux éléments lors de la compilation, tout est plus simple :

```
int [][] tableau = { { 1, 2, 3, 4, 5 }, { 6, 7, 8, 9, 10 } };
```

2.8.1. Exercices

2.8.1.1. Tri à bulles

Créez une classe permettant de trier un tableau (pas une collection du framework Java) d'entiers en utilisant le tri à bulles.

Le principe du tri à bulles est de parcourir le tableau pour comparer chaque élément avec celui qui le suit. Nous permutons les deux éléments si le suivant est inférieur au premier. Nous recommençons à parcourir le tableau tant qu'il existe des éléments à trier.

La classe *TriABulles* contiendra une seule méthode dans laquelle l'algorithme sera implanté. Le tableau sera déclaré comme suit :

```
int [] tableau = { 56, 45, 3, 67, 67, 45, 78, 26 };
```

[Corrigé.](#)

2.8.1.2. Création d'une matrice

Créez une matrice de 10x10 en utilisant les tableaux de base. Remplissez les cellules en utilisant une valeur aléatoire comprise entre 0 et 1.

Imprimez ensuite cette matrice en affichant 10 nombres par ligne.

[Corrigé.](#)

2.8.1.3. Tri d'un tableau de chaînes de caractères

Créez un tableau de 5 chaînes de caractères. Ajoutez dans l'ordre au tableau les chaînes suivantes :

- Python
- Basic
- Perl
- C++
- Java

Triez le tableau puis imprimez le résultat sur l'écran.

[Corrigé.](#)

2.8.1.4. Extraction de mots

Ecrivez un programme qui initialise une variable de type *String* avec un texte de votre choix. Vous devez extraire tous les mots du texte et les trier par ordre alphabétique. Imprimez la liste de mots. Vous pouvez utiliser le même algorithme que dans l'exercice précédent (le tri à bulles).

[Corrigé.](#)

2.9. Le passage des arguments à une méthode

En Java, le passage des arguments est toujours réalisé par valeur. Une méthode reçoit donc une copie de l'argument. Voici un exemple montrant le phénomène sur des types de base :

```
public class TestPassageArguments {
    public static void main (String Arguments []) {
        int a = 10;
        int b = 20;

        System.out.println("a: " + a + ", b: " + b);
        echangeAetB(a, b);
        System.out.println("a: " + a + ", b: " + b);
    }

    public void echangeAetB(int a, int b) {
        int tmp = a;
        a = b;
        b = tmp;
        System.out.println("a: " + a + ", b: " + b);
    }
}
```

L'affichage écran de ce programme montre que l'échange entre *a* et *b* n'a pas eu lieu hors de la méthode *echangeAetB*. Par contre, si nous passons des objets comme arguments, la méthode travaillera sur les valeurs réelles et non pas une copie, car les objets sont passés par référence, c'est-à-dire par adresse. La méthode recevra une copie de la référence, mais cela n'aura aucune importance, car la valeur de l'adresse pointée par la référence sera toujours la même. Le même exemple que le précédent :

```
class Entier {
    public int entier = 0;
    public String toString() { return Integer.toString(entier); }
}

public class TestPassageArguments {
    public static void main (String Arguments []) {
        Entier a = new Entier();
        a.entier = 10;

        Entier b = new Entier();
        b.entier = 20;

        System.out.println("a: " + a + ", b: " + b);
        echangeAetB(a, b);
    }
}
```

```

        System.out.println("a: " + a + ", b: " + b);
    }

    public static void echangeAetB(Entier a, Entier b) {
        int tmp = a.entier;
        a.entier = b.entier;
        b.entier = tmp;
        System.out.println("a: " + a + ", b: " + b);
    }
}

```

Cette fois, les valeurs ont bien été échangées, car la méthode a travaillé sur une copie des adresses mémoires des objets *a* et *b*. Il nous est facile de comprendre que cela revient au même.

Nous pouvons rencontrer un problème avec un certain type de classe : les classes immutables. Ces classes ne peuvent pas être modifiées. Pour les connaître, nous devons parcourir la documentation du framework Java. Par exemple, les classes *Integer*, *String*, etc. sont immutables.

L'exemple suivant met en évidence le problème :

```

public class TestPassageArguments {
    public static void main (String Arguments []) {
        Integer a = new Integer(10);
        Integer b = new Integer(20);

        System.out.println("a: " + a + ", b: " + b);
        echangeAetB(a, b);
        System.out.println("a: " + a + ", b: " + b);
    }

    public static void echangeAetB(Integer a, Integer b) {
        int tmp = a;
        a = b;
        b = tmp;
        System.out.println("a: " + a + ", b: " + b);
    }
}

```

2.10. Les variables locales

Une variable locale est une variable déclarée dans une méthode. Le corps de la méthode définit ce qu'on appelle la « portée » de la variable. Lorsque le programme sort de la méthode, la variable devient inaccessible. Cela entraîne les conséquences suivantes :

- La variable est un type de base : elle est immédiatement détruite et le contenu est perdu.
- La variable est une référence sur un objet : la variable locale est détruite, mais l'objet qu'elle pointait continue d'exister si cet objet est atteignable à un autre endroit du programme : c'est-à-dire qu'il est encore référencé par une variable valide. Si cet objet n'est plus référencé, le ramasse-miettes le supprimera dans un moment plus ou moins lointain.

Initialisation des variables locales

Les variables locales ne sont pas initialisées à 0 pour les types de base numériques ou à *null* pour les références. Une erreur sera générée par le compilateur lors de la première instruction qui essaiera de lire le contenu de la variable. Nous devons donc initialiser correctement toutes les variables locales lors de leur déclaration.

2.11. Les packages

Au cours d'un développement, le nombre de classes peut s'accroître rapidement. Il est rapidement nécessaire de les organiser en packages pour :

- Organiser les classes par catégorie afin de les localiser plus rapidement.
- Gérer les conflits de noms : deux classes peuvent avoir le même nom si elles sont situées dans deux packages différents.
- Etablir des contrôles d'accès aux classes, à leurs méthodes et leurs attributs entre les packages.

Le framework Java contient de nombreux packages, comme par exemple le package *java.io* pour les classes destinées aux entrées/sorties. Toutes les classes relatives aux entrées/sorties ont été regroupées dans ce package. Pour pouvoir utiliser une classe située dans ce package, nous utilisons le mot clé *import*.

```
import java.io.*;
```

Note

Le package *java.lang*

Avant de pouvoir utiliser une classe située dans un package, nous devons utiliser le mot clé *import*. Pourtant, nous avons déjà utilisé certaines classes comme les classes *Integer* et *String* sans utiliser ce mot clé. La raison est que les classes situées dans le package *java.lang* sont directement accessibles sans que nous ayons besoin de les importer.

Les packages Java sont organisés en utilisant le système de fichiers. L'arborescence d'un package est donc constituée de répertoires dans lesquels sont stockés les fichiers *.java* contenant les classes.

La création d'un package consiste à créer un répertoire. Le chemin vers ce répertoire devra être ajouté à la variable d'environnement *CLASSPATH* pour que le package soit accessible par le compilateur et la JVM.

Sous Unix, si le répertoire qui contient nos packages est */home/jmcaricand/alao/packages*, nous modifions la variable *CLASSPATH* de cette manière :

```
export CLASSPATH=/home/jmcaricand/alao/packages:$CLASSPATH
```

Sous Windows, si le répertoire est *C:\Documents and settings\jmcaricand\alao\packages* :

```
set CLASSPATH=c:\Documents and settings\jmcaricand\alao\packages;%CLASSPATH%
```

Si nous voulons créer le package *utils*, nous créons le répertoire :

```
# cd /home/jmcaricand/alao/packages
# mkdir utils
```

Dans le répertoire */home/jmcaricand/alao/packages/utils*, nous créons la classe *Test*. La première ligne du fichier *Test.java* sera *package utils;* .

```
package outils;

public class Test {
    public void message() {
        System.out.println( "Test.message" );
    }
}
```

Afin de conserver une compatibilité entre les systèmes d'exploitation, il est conseillé de toujours nommer les packages en minuscules.

Un package peut faire partie d'un autre package :

```
# mkdir outils/patterns
```

Dans le répertoire *outils/patterns*, nous créons une classe *Singleton* :

```
package outils.patterns;

public class Singleton {
    ...
}
```



Nommer les packages

Un conseil de nommer les packages de manière à ne pas entrer en conflit avec d'autres packages. Une solution consiste à ajouter au nom du package le nom de domaine DNS de notre société. Par exemple, un package Graphique créé par une société possédant le nom de domaine *societe.com* pourrait être *com.societe.graphique*. L'utilisation du nom de domaine devrait être suffisante pour éviter les conflits, car il est théoriquement unique.

Le framework Java possède de nombreux packages. Les plus utilisés sont :

17. Tableau 3.1. Les packages Java

Package	Description
java.lang	Classes de base du langage
java.util	Divers outils (dates, collections...)
java.io	Classes concernant les entrées sorties
java.awt	Interface graphique
java.awt.event	Gestion des événements de l'interface graphique

2.12. Contrôle d'accès en Java

Le contrôle d'accès permet d'appliquer le concept d'encapsulation cher à la programmation orientée objet. Plusieurs types de contrôles peuvent être réalisés par Java en fonction des spécificateurs utilisés et de l'emplacement des classes dans les packages.

Nous avons déjà utilisé des spécificateurs d'accès aux attributs et aux méthodes dans l'écriture de certaines classes. Les spécificateurs d'accès sont les suivants :

- *public* : le membre est accessible sans restrictions.
- *protected* : le membre est accessible par toutes les classes de son package et toutes les classes dérivées.
- *private* : le membre n'est accessible par aucune autre classe.
- aucun spécificateur (« *friendly* ») : le membre est accessible uniquement des classes situées dans le même package.

Les classes possèdent également des spécificateurs d'accès :

- *public* : la classe est visible de partout. Elle doit être située dans un fichier qui porte son nom.
- aucun spécificateur (« *friendly* ») : la classe est visible uniquement dans son package.

Il nous est possible de redéfinir le spécificateur d'accès à un membre lors d'une relation d'héritage. Toutefois, la redéfinition doit aller dans le sens de la restriction, pas de l'ouverture vers l'extérieur.

2.13. Les exceptions

Certaines erreurs surviennent dans le déroulement d'un programme. Ces erreurs ne sont pas nécessairement le résultat du codage. Néanmoins, l'interception des erreurs non prévues est un plus qui est possible en Java grâce au mécanisme des exceptions. L'exemple le plus courant est la division d'un nombre par zéro. Dans les langages procéduraux traditionnels, ce type d'erreur doit être prévu par le développeur sous peine d'un arrêt brutal du programme. Dans la plupart des cas, le contrôle est fait à la saisie du diviseur. Les langages à objets modernes intègrent des outils qui nous permettent de contrôler de manière propre ces erreurs. C'est le rôle des exceptions.

Une exception est un événement anormal qui survient dans le déroulement du programme. Nous avons alors les possibilités suivantes :

- ne rien faire : le programme s'arrête avec un message d'erreur précis sur la cause de l'exception.
- gérer l'exception dès qu'elle survient pour en corriger la cause.
- laisser l'exception se propager pour être traitée ailleurs dans le programme.

```
public class Division
{
    public static void main (String[] args)
    {
        int numerator = 4;
        int denominator = 0;

        int quotient = numerator / denominator;

        System.out.printf ("%d / %d = %d\n", numerator, denominator, quotient);

        System.out.println ("Fin du programme");
    }
}
```

Dans le programme précédent, nous allons avoir un problème : le dénominateur à la valeur zéro. La ligne

```
int quotient = numerator / denominator;
```

va provoquer une erreur qui va interrompre le programme, car la division par zéro va générer une exception. Nous ne verrons donc pas l'affichage du texte « Fin du programme ».

La JVM va quand même nous aider dans la correction de l'erreur en nous indiquant la cause de celle-ci. Le programme s'arrête sur une exception de type *ArithmeticException* en nous indiquant également le numéro de ligne.

Nous pouvons prévoir l'erreur avec un traitement procédural :

```
public static void main (String[] args)
{
    int numerator = 4;
    int denominator = 0;

    if (denominator != 0)
    {
        int quotient = numerator / denominator;
        System.out.printf ("%d / %d = %d\n", numerator, denominator, quotient);
    }
    else
    {
        System.out.println ("Vous ne pouvez pas faire une division par zéro
!");
    }

    System.out.println ("Fin du programme");
}
```

Cette manière de procéder ne permet pas d'intercepter toutes les erreurs possibles. Certaines erreurs systèmes ne pourront pas de plus être traitées avec un *if-else*. Heureusement, les langages à objets intègrent tous un traitement des erreurs plus générique : une gestion des erreurs à travers des exceptions.

Une exception est un objet qui transporte des informations sur un type d'erreur particulier, jusqu'à un point du programme appelé un gestionnaire d'exceptions. Nous devons seulement indiquer au compilateur quelle partie du programme nous désirons surveiller. Le bloc d'instructions à surveiller est défini dans un *try { ... }*. Juste après ce bloc *try*, nous trouvons les gestionnaires d'exceptions. Ces gestionnaires sont matérialisés par des blocs *catch(type d'exception) { ... }*. Nous écrivons « les gestionnaires », car nous pouvons traiter différemment les exceptions que nous aurions pu rencontrer dans le *try { ... }*.

Une exception est un objet qui hérite de la classe *java.lang.Throwable*. Seules les classes qui héritent de cette classe pourront être envoyées par une instruction en erreur à l'aide de l'instruction *throw*, puis être capturées par l'instruction *catch*. L'ensemble du framework Java est basé sur l'utilisation des exceptions et la connaissance de celles-ci est donc un impératif dans ce langage.

Reprenons le programme précédent en utilisant les exceptions :

```
public static void main (String[] args)
{
    int numerator = 4;
    int denominator = 0;
```

```

try
{
    int quotient = numerator / denominator;
    System.out.printf ("%d / %d = %d\n", numerator, denominator, quotient);
}
catch (ArithmeticException exception)
{
    System.out.println ("Vous ne pouvez pas faire une division par zéro
!");
}

    System.out.println ("Fin du programme");
}

```

Nous mettons les instructions à surveiller dans un bloc *try* { ... } puis indiquons au compilateur qu'il faut traiter les exceptions du type *ArithmeticException* d'une manière spécifique. La ligne

```
System.out.println ("Fin du programme");
```

ne fait pas partie du bloc *catch* car elle devra toujours être exécutée.

Nous pouvons observer que la gestion des erreurs est plus lisible qu'un *if-else* et surtout plus sûre. Pour être certains de capturer toutes les exceptions possibles, nous pouvons ajouter un *catch* supplémentaire :

```

public static void main (String[] args)
{
    int numerator = 4;
    int denominator = 0;

    try
    {
        int quotient = numerator / denominator;
        System.out.printf ("%d / %d = %d\n", numerator, denominator, quotient);
    }
    catch (ArithmeticException exception)
    {
        System.out.println ("Vous ne pouvez pas faire une division par zéro
!");
    }
    catch (Exception exception)
    {
        System.out.println ("Une exception imprévue !");
    }

    System.out.println ("Fin du programme");
}

```

En ajoutant des *catch* successifs, nous devons prendre soin de traiter les exceptions les plus spécialisées en premier pour terminer avec les plus génériques. En effet, l'instruction *catch* se base sur la hiérarchie des classes d'exceptions pour appliquer les traitements. Un seul bloc *catch* est exécuté à la suite d'un *try*.

Observons la hiérarchie des classes d'exceptions de Java :

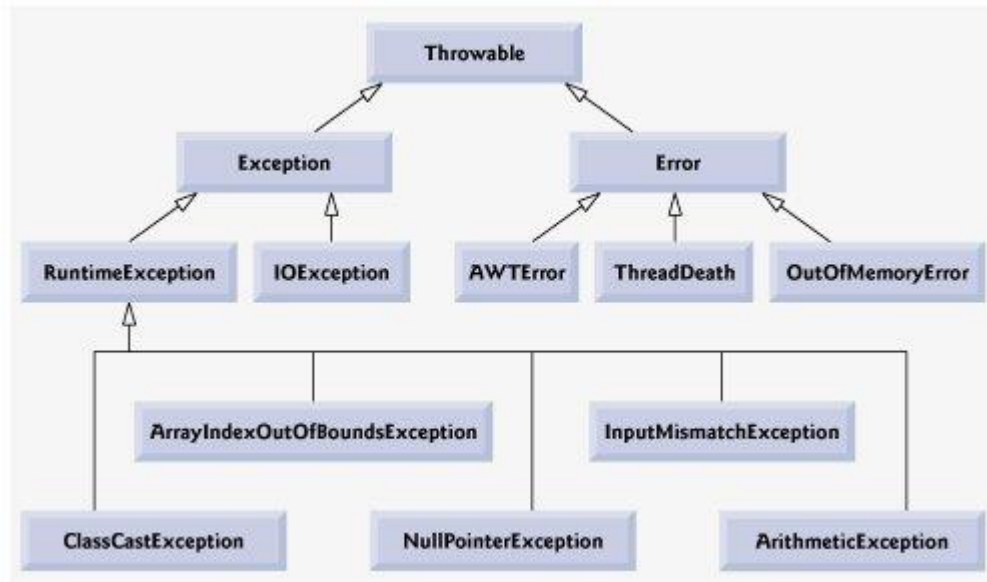


Diagramme UML - Relation d'association

Toutes les classes d'exceptions héritent de *Throwable*. Nous avons ensuite deux branches distinctes :

- *Error* : les exceptions de type *Error* sont graves et ne peuvent pas être traitées par l'application. Le programme est stoppé immédiatement.
- *Exception* : les exceptions de type *Exception* doivent être capturées par un gestionnaire d'exceptions. D'ailleurs, le compilateur impose que les méthodes pouvant envoyer ces exceptions soient appelées dans un bloc *try*. Il génère une erreur de compilation si ce n'est pas le cas.

Située dans la branche *Exception*, nous avons également une classe *RuntimeException*. Les méthodes envoyant ce type d'exception n'ont pas besoin d'être dans un bloc *try*. Ces exceptions sont générées, par exemple, lors d'un accès à un index non valide d'un tableau, etc.

Les exceptions métiers

Les exceptions métiers, c'est-à-dire les exceptions que nous créons pour nos besoins, sont en général dérivées de la classe *Exception*.

2.13.1. Les exceptions métiers

Une exception métier une classe d'exception qui doit transporter des informations sur des erreurs survenant dans une application que nous développons. Avant de pouvoir utiliser une exception métier, nous devons créer une classe spécialisée comme suit :

- Elle doit hériter de la classe *Exception*. De cette manière, elle devra être capturée.
- Le constructeur de notre classe doit prendre au minimum un argument de type *String* qui décrit le type d'erreur rencontrée.
- Elle doit, si nécessaire, surcharger la méthode *getMessage* qui permet de récupérer le message d'erreur transporté.
- Elle doit contenir les méthodes et les attributs pouvant être utiles au traitement de l'erreur.

Une fois définie, la classe d'exception doit être « lancée » lorsque nous rencontrons une situation exceptionnelle dans le déroulement de notre application. Pour lancer une exception, nous utilisons

l'instruction *throws*. Cette instruction attend un objet du type *Throwable*. Le compilateur refusera de compiler si la classe passée à *throws* ne respecte pas cette condition.

Une autre remarque s'impose au sujet de l'instruction *throws* : une méthode susceptible de lancer une exception doit posséder un spécificateur *throws* suivi de la liste des exceptions qui peuvent être lancées.

2.13.2. L'instruction *finally*

Le modèle objet de C++ permet l'utilisation des destructeurs. Un destructeur est une méthode spéciale qui est appelée lors de la destruction d'un objet. En général, nous libérons toutes les ressources allouées par l'objet dans cette méthode. Cette manière de faire n'est pas possible en Java, car la notion de destructeur n'existe pas. Il existe bien une méthode *finalize* mais elle est peu utile. D'ailleurs, le framework Java y fait rarement appel. Le problème avec Java, c'est que la destruction des objets est de la responsabilité du ramasse-miettes. Nous ne pouvons pas savoir à quel moment un objet sera réellement libéré⁴. Impossible donc de restituer les ressources systèmes dans la méthode *finalize*.

Le problème devient épineux lorsqu'un objet ouvre un fichier dans un bloc *try* et qu'une exception est levée. Le point d'exécution du programme peut ne jamais exécuter les instructions qui devraient fermer le fichier ouvert par l'objet. Heureusement, nous avons à notre disposition l'instruction *finally*. Le bloc *finally* est placé après le ou les blocs *catch* et surtout, il est toujours exécuté.

finally sert donc principalement à remettre en état la cohérence d'un traitement qui aura été perturbé par la levée d'une exception.

2.13.3. Exemple complet

Mettons en pratique l'ensemble des remarques à travers un petit exemple qui reprend les principales remarques vues précédemment.

```
class AgeNonValideException extends Exception {
    public AgeNonValideException( String message ) {
        super( message ) ;
    }
}

class Personne {
    Integer age = new Integer( 0 );

    public Personne() {
    }

    public void setAge( Integer age ) throws AgeNonValideException {
        if ( age < 0 || age > 130 ) {
            throw new AgeNonValideException( "L'age \"" + age + "\" est en " +
                                                "dehors des limites autorisées" );
        }

        this.age = age;
    }
}

public class UtiliserException {
```

⁴ Il peut ne jamais être libéré si la mémoire est encore vaste.

```

public static void main( String [] args ) {
    Personne p = new Personne();

    try {
        p.setAge( Integer.valueOf( 150 ) );
        System.out.println( "Age modifié pour p" );
    }
    catch ( AgeNonValideException e ) {
        System.out.println( "Une exception a été levée pour la modification
de l'age de p :\n" +
                                "Le message transporté par l'exception est : "
+ e.getMessage() );
    }
    finally {
        System.out.println( "Le bloc finally est toujours exécuté !" );
    }
}
}

```

Nous pouvons remarquer que la méthode *setAge* annonce qu'elle est susceptible de lever une exception. Dans cette méthode, l'instruction *throw* lance une exception créée in-situ avec un message qui sera transporté jusqu'au gestionnaire chargé de la traiter. La méthode *setAge* doit être dans un bloc *try*, car l'exception *AgeNonValideException* dérive de la classe *Exception*. Le compilateur refusera de compiler le programme si ce n'est pas le cas. Dans le bloc *catch*, nous affichons la raison de l'erreur. Le bloc *finally* est exécuté, qu'une exception soit levée ou non.

2.14. Mot-clé this

Le mot-clé *this* désigne une référence sur l'instance courante de la classe (sur l'objet courant). Voici un exemple :

```

public class Chien {
    private String couleur;
    ...

    public void foo() {
        String cou = this.couleur;
        ...
    }
}

```

3. Maven⁵

3.1 Qu'est-ce que Maven et pourquoi faut-il l'utiliser ?

Maven permet de faciliter et d'automatiser certaines tâches de la gestion d'un projet Java. Il permet notamment :

- D'automatiser certaines tâches : compilation, tests unitaires et déploiement des applications qui composent le projet
- De gérer des dépendances vis à vis des bibliothèques nécessaires au projet

⁵ Le chapitre sur *Maven* est tiré de « [Wikibooks / Introduction à Apache Maven](#) ».

- De générer des documentations concernant le projet

Pour gérer les dépendances du projet vis à vis de bibliothèques, Maven utilise un ou plusieurs dépôts (appelés repositories) qui peuvent être locaux (.maven/repository) ou distants (<http://www.ibiblio.org/maven> par défaut)

Maven est extensible grâce à un mécanisme d'extension qui permet d'ajouter des fonctionnalités.

Maven comparé à Ant

Au premier abord, il est facile de croire que Maven fait double emploi avec Ant. Ant et Maven sont tous les deux développés par le groupe Jakarta, ce qui prouve bien que leur utilité n'est pas aussi identique. Ant dont le but est d'automatiser certaines tâches répétitives est plus ancien que Maven. Maven propose non seulement ces fonctionnalités mais en propose de nombreuses autres.

Les différentes versions de Maven

Plusieurs versions majeures de Maven ont été diffusées :

- version 1.0, juillet 2004
- version 1.1, juin 2007
- version 2.0, octobre 2005
- version 2.1, mars 2009
- version 3.0, octobre 2010
- version 3.1, juillet 2013

Maven 2 est très différent de Maven 1.

Maven 3 est compatible avec Maven 2 et apporte notamment de meilleures performances.

3.2 Mettre en place Maven

Installez Maven sur votre machine

Ici, la procédure diffère selon le système d'exploitation que vous utilisez. L'installation sous Linux est extrêmement simple, l'installation sous Windows l'est moins.

Installation sous Windows

Il faut télécharger [<http://maven.apache.org/download.html>] la version la plus récente de Maven sur le site officiel et la dézipper.

Il suffit de placer les fichiers dans un répertoire qui sera spécifié dans la variable d'environnement M2_HOME.

Ensuite, définir les variables d'environnement correspondantes, par exemple :

- JAVA_HOME = C:\Program Files\Java\jdk1.8.0_181\ (à adapter à votre environnement)
- M2_HOME = C:\java\apache-maven-3.5.4\ (à adapter à votre environnement)
- path = C:\java\apache-maven-3.5.4\bin (à adapter à votre environnement)

Installation sous GNU/Linux

Rien n'est plus simple : utilisez votre gestionnaire de paquets.

Installation sous Mac OS X

Avec le gestionnaire de paquets Homebrew

```
$ brew install maven
```

où macPorts

```
$ sudo port install maven3
```

Testez votre installation

```
$ mvn --version
```

Configurez Maven

Dans votre répertoire personnel, vous trouverez un dossier « .m2 » contenant un fichier « settings.xml » qui va vous permettre de configurer Maven.

3.3 Créez un premier projet

Maven se base sur les conventions pour trouver tous les composants de vos projets, suivez-les et Maven se débrouillera sans aucune configuration.

Arborescence d'un projet Maven

Voici l'arborescence minimum d'un projet Maven.

```
mon_projet
```

Un dossier qui va contenir toutes les données du projet

```
mon_projet/src
```

Ce dossier va contenir tous les fichiers sources créés par les développeurs pour le projet

```
mon_projet/src/main  
mon_projet/src/main/java
```

Contient les codes sources Java, rangés selon la hiérarchie des packages. (par exemple, mon_projet/src/main/java/org/wikibooks/fr/Exemple.java pour la classe org.wikibooks.fr.Exemple)

```
mon_projet/src/main/resources
```

Les ressources nécessaires au code Java. Fichiers de configuration, icônes, fichiers de données...

```
mon_projet/src/test
```

Contient toutes les données nécessaires pour tester l'application

```
mon_projet/src/test/java
```

Contient les codes sources des tests

```
mon_projet/src/test/resources
```

Les ressources qui sont utilisées pour les tests

```
mon_projet/pom.xml
```

Un fichier XML qui décrit le projet pour permettre à Maven d'interagir avec lui.

Le « Project Object Model » : fichier pom.xml

Tous les projets Maven sont identifiés de façon unique. Pour cela, un projet comporte 3 valeurs :

Un groupId

Il désigne l'organisation responsable de ce projet. Il arrive souvent qu'on suive les conventions sur les noms de package pour définir le groupId. Votre groupId doit être propre à votre organisation et être composé d'un nom de domaine dont vous avez le contrôle pour être sûr qu'aucun autre projet qui n'est pas de vous ne porte le même groupId. Bon exemple, si vous travaillez dans une entreprise, utilisez le nom de domaine de votre entreprise (com.maboite), si vous êtes étudiants, utilisez le domaine de votre formation (fr.monuniversite.moncours), si c'est un logiciel libre, utilisez l'adresse du site du projet ou de la forge, etc.

Un artifactId

Il identifie le projet lui même : pour faire simple, c'est le nom que portera le fichier .jar qui sera généré par la construction du projet. Il faut donc veiller à ce que son nom soit sans ambiguïté. Bons exemples : junit, commons-io, http-client ; mauvais exemples : test, core, module.

Une version

Pour finir d'identifier une application ou une bibliothèque, on ajoute une troisième composante, la version.

Commençons par un pom minimal :

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
  <!-- Toujours utiliser 4.0.0 pour maven 2.x -->
  <modelVersion>4.0.0</modelVersion>

  <!-- groupId, artifactId et version actuelle -->
  <groupId>org.wikibooks.fr</groupId>
  <artifactId>projet-minimal</artifactId>
  <version>1.0</version>
```

```

<!-- la façon dont sera packagée l'application -->
<packaging>jar</packaging>

<!-- un nom pour le projet -->
<name>Un projet d'exemple pour le livre Développer en Java de
Wikibooks</name>

<!-- l'adresse du site qui présente cette application -->
<url>http://fr.wikibooks.org/wiki/Développer_en_Java/Introduction_à_Apache_Mave
n</url>
</project>

```

3.4 Gérez les dépendances de votre projet

Maven gère toutes les dépendances de vos projets. Pour cela, il stocke toutes les bibliothèques de vos projets dans un dépôt local⁶. À chaque fois que vous demanderez à Maven de construire le projet, Maven ira chercher toutes les dépendances dans votre dépôt local.

Si Maven ne trouve pas la dépendance dans votre dépôt local, il ira automatiquement la télécharger depuis un dépôt sur Internet et l'ajoutera à votre dépôt local. Au début, votre dépôt est vide, donc Maven téléchargera beaucoup de choses lors des premières utilisations.

Votre dépôt local est bien rangé, Maven organise tout et range toutes les bibliothèques selon le groupId, l'artifactId et les versions. À tout moment, vous pouvez supprimer l'intégralité de votre dépôt local pour faire le ménage. Maven ne re-téléchargera que le nécessaire.

Déclarer une dépendance

Si votre application a besoin d'une dépendance, c'est simple, il suffit d'ajouter au POM la dépendance : son groupId, artifactId et version. Par exemple, pour que les développeurs bénéficient de l'API commons lang, il suffit d'ajouter dans le POM :

```

<!-- ... -->

<dependencies>

  <dependency>
    <groupId>commons-lang</groupId>
    <artifactId>commons-lang</artifactId>
    <version>2.5</version>
  </dependency>

</dependencies>

```

Pour compiler ou lancer le projet, Maven ira chercher la dépendance dans le dépôt local, ou sur Internet si besoin.

Les différents « scope »

⁶ Vous pourrez le trouver dans votre dossier personnel, dans ".m2/repository"

Une dépendance a une portée (scope), c'est à dire qu'elle est utilisée pour certaines tâches. Il existe les portées (scopes) suivantes :

compile

c'est le scope par défaut. La dépendance est utilisée pour compiler le code et est nécessaire à son exécution. Lorsque l'application sera construite par Maven, cette dépendance sera ajoutée dans le fichier jar généré.

test

cette dépendance est nécessaire seulement pour la compilation et l'exécution des tests. Quand Maven construira l'application, il n'ajoutera pas cette dépendance au jar généré étant donné que cette bibliothèque n'est pas utilisée dans l'application principale mais seulement dans les tests (remarque : les tests bénéficient, eux, des dépendances compile).

runtime

la dépendance n'est pas disponible à la compilation mais est fournie au lancement de l'application. Par exemple, un pilote de base de données (JDBC).

provided

la dépendance ne sera pas jointe à l'application mais sera fournie quand elle sera déployée. Par exemple, une application peut dépendre d'un élément qui ne lui sera fourni que lorsqu'elle sera déployée dans le serveur d'application. Comme il y a plusieurs serveurs d'application différents, on confie à ce dernier le soin de fournir la dépendance.

Par la suite, nous aurons l'usage des ces différents scopes.

Comment trouver des bibliothèques ?

Lorsque Maven cherche une dépendance sur Internet, il s'oriente vers un dépôt principal où se trouvent l'essentiel des bibliothèques Java disponibles et qui s'appelle « central ». Et une application en ligne permet de parcourir son contenu et de faire des recherches par groupId, artifactId et version [archive]. Une fois que vous avez trouvé la bibliothèque qui remplit vos besoins, l'application peut vous donner le petit code XML à ajouter au POM du projet.

3.5 Un POM d'exemple complet

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <!-- groupId, artifactId et version actuelle -->
  <groupId>org.wikibooks.fr</groupId>
  <artifactId>projet-minimal</artifactId>
```

```

<version>1.0</version>

<!-- la façon dont sera packagée l'application -->
<packaging>jar</packaging>

<!-- un nom pour le projet -->
<name>Un projet d'exemple pour le livre Développer en Java de
Wikibooks</name>

<!-- l'adresse du site qui présente cette application -->
<url>http://fr.wikibooks.org/wiki/Développer_en_Java/Introduction_à_Apache_Mave
n</url>

<dependencies>
  <dependency>
    <groupId>commons-logging</groupId>
    <artifactId>commons-logging</artifactId>
    <version>1.1.1</version>
    <scope>compile</scope>
  </dependency>
  <dependency>
    <groupId>log4j</groupId>
    <artifactId>log4j</artifactId>
    <version>1.2.16</version>
    <scope>runtime</scope>
  </dependency>

  <dependency>
    <groupId>commons-io</groupId>
    <artifactId>commons-io</artifactId>
    <version>1.4</version>
  </dependency>

  <!-- test -->
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.8.1</version>
    <scope>test</scope>
  </dependency>
</dependencies>

<properties>
</properties>

<build>
</build>

</project>

```

3.6 Profitez des fonctionnalités de Maven

Tout est en place, le projet est prêt. On peut maintenant passer à l'action et profiter des nombreuses possibilités offertes par Maven.

Construire le projet

La construction d'un projet se passe en différentes phases successives qui décrivent le cycle de vie (Build Lifecycle) :

validate

vérifie que le POM est correct et complet

compile

compile les sources du projet qui se trouvent dans src/main/java et place les fichiers compilés (.class) dans target

test

compile les tests qui se trouvent dans src/test/java et lance l'intégralité des tests, les tests compilés sont placés dans target

package

génère un package de l'application dans target/artifactId-version.jar

install

utilise le package généré à la phase package et l'ajoute au dépôt local. À utiliser pour une bibliothèque : elle peut dès lors être utilisée comme dépendance pour un autre projet.

deploy

déploie l'application (varie selon la configuration)

Une autre phase est :

clean

détruit le répertoire target

Voici quelques exemples d'utilisation :

```
# on peut lancer une phase avec mvn phase, par exemples
mvn compile # exécute la phase validate puis la phase compile
mvn package # fait appel à toutes les phases précédentes

# on peut donner plusieurs phases en paramètres, par exemple
mvn clean test # exécute la phase clean puis la phase test

# Enfin, quelque-soit la phase, on peut demander à Maven de sauter les tests,
pour gagner du temps
mvn install -Dmaven.test.skip # saute la compilation et l'exécution des tests
# ou
mvn install -DskipTests # compile les tests mais saute l'exécution
```

Il y a d'autres phases intermédiaires mais nous avons choisi de ne vous présenter que celles qui sont utilisées 80% du temps.

Exploiter les nombreuses possibilités offertes par les extensions Maven

L'outil mvn peut faire appel à des extensions. Chaque extension est proposée avec un ensemble d'actions possibles appelées goal. On utilise alors mvn extension:goal.

Pour la plupart des extensions, il existe un goal help. Par exemple, mvn javadoc:help devrait donner l'aide pour l'extension javadoc et tous les goals que propose cette extension.

Générer un site internet pour le projet

```
mvn site:site
```

Générer la javadoc

La documentation sera générée dans target/site/apidocs.

```
mvn javadoc:javadoc
```

Packager l'application

Générer un jar avec toutes les dépendances incluses. Utile pour proposer une version complète standalone de l'application.

```
mvn assembly:assembly -DdescriptorId=jar-with-dependencies
```

Générer un jar avec seulement les sources, idéal pour distribuer le code à des développeurs pour qu'ils l'intègre à leur IDE.

```
mvn assembly:assembly -DdescriptorId=src
```

Exécuter une classe java

Si vous voulez exécuter le programme, vous pouvez faire

```
mvn package
cd target
java -jar mon-artifact.jar
```

Cela ne fonctionnera pas étant donné qu'il faut ajouter au CLASSPATH tout un tas de fichier jar qui se trouvent dans votre dépôt local. La plug-in exec vous permet d'exécuter un programme principal.

```
mvn exec:java -Dexec.mainClass=org.wikibooks.fr.ClassePrincipale -
Dexec.args="argument1 argument2"
# attention : -Dexec.args="" passe l'argument "" (chaîne vide) à l'application
```

On peut aussi ajouter les informations sur le nom de la classe principale, les arguments et les réglages systèmes dans le POM.

Surveiller les mises à jour des dépendances

À tout moment, vous pouvez vérifier la disponibilité de nouvelles versions de vos dépendances avec

```

mvn versions:display-dependency-updates
[INFO] [versions:display-dependency-updates {execution: default-cli}]
[INFO] The following dependencies in Dependency Management are using the newest
version:
[INFO] commons-daemon:commons-daemon ..... 1.0.1
[INFO] commons-io:commons-io ..... 1.4
[INFO] commons-lang:commons-lang ..... 2.5
[INFO] commons-logging:commons-logging ..... 1.1.1
[INFO] junit:junit ..... 4.8.1
[INFO] log4j:log4j ..... 1.2.16
[INFO]
[INFO] The following dependencies in Dependency Management have newer versions:
[INFO] org.mortbay.jetty:jetty ..... 6.1.9 -> 6.1.25
[INFO] org.quartz-scheduler:quartz ..... 1.8.3 -> 1.8.4

```

Ce plug-in versions permet également de mettre à jour automatiquement les numéros de versions dans un projet.

4. Les fichiers JAR

Une application Java peut être composée de plusieurs dizaines de fichiers *.class*. Cette prolifération de fichiers nuit au déploiement de l'application. Pour résoudre ce problème, Java fournit la possibilité de créer des fichiers archives compressés au format ZIP. Ces fichiers ont l'extension *.jar*⁷. Ces fichiers peuvent contenir une application complète ou un ensemble de classes formant une bibliothèque.

Pour créer une archive JAR, le JDK fournit l'utilitaire *jar* ou *jar.exe* en fonction du système d'exploitation utilisé.

La commande *jar* fonctionne avec un ensemble d'options que nous pouvons afficher avec l'option *--help* :

```
# jar --help
```

Pour créer une archive contenant toutes les classes situées dans le répertoire *mesclasses*, nous pouvons utiliser la commande suivante :

```
# jar cfm mesclasses.jar manifest.txt mesclasses/*.class
```

Le fichier *manifest* contient des informations utiles sur le contenu de l'archive. Ces informations peuvent être lues et utilisées par certains outils lors du déploiement de l'archive. Si nous ne spécifions pas de fichier *manifest*, le programme *jar* en crée un avec des informations par défaut.

Alors que le fichier *manifest* n'est pas forcément utile pour la création d'une bibliothèque de classes, il doit être correctement renseigné lorsque nous archivons une application contenant plusieurs fichiers *.class*. En effet, nous devons indiquer dans ce fichier quel fichier *.class* contient la fonction *main*, car cette classe représente le point d'entrée dans le programme.

Une ligne contenant la clé *Main-Class*: devra être présente dans le fichier. Prenons comme exemple l'application *TestJAR*. Cette application est constituée d'un fichier contenant la classe *Compte* et d'un

⁷ JAR est l'acronyme de Java Archive.

fichier *TestJAR.class* contenant la classe *TestJAR*. Cette classe est la classe de démarrage du programme. Voici le listing des deux fichiers sources :

Exemple 4.1. JAR - Compte.java

```
public class Compte {
    String nom;
    Double solde;

    public Compte() {
    }

    public Compte( String nom, Double solde ) {
        this.nom = nom;
        this.solde = solde;
    }

    public String toString() {
        return this.getClass().getName() + ", " +
            nom + ", " + solde;
    }
}
```

Exemple 4.2. JAR - TestJAR.java

```
public class TestJAR {
    public static void main( String [] args ) {
        Compte compte1 = new Compte( "jmcaricand", new Double( 1000.00 ) );
        Compte compte2 = new Compte( "sbernard", new Double( 23. ) );

        System.out.println( "Compte 1 :\n" + compte1 + "\n\n" );
        System.out.println( "Compte 2 :\n" + compte2 + "\n\n" );
    }
}
```

Exemple 4.3. JAR - Manifest.txt

```
Manifest-Version: 1.0
Created-By: Jean-Michel Caricand
Main-Class: TestJAR
```

[Note] Le fichier Manifest

Le contenu du fichier manifest doit être encodé en UTF8. Cet encodage est particulièrement important si nous avons des caractères accentués dans le fichier.

Nous créons notre archive :

```
# jar cfm testjar.jar Manifest.txt *.class
```

En utilisant un outil comme WinZIP, nous pouvons voir que le fichier Manifest est inclus dans l'archive et placé dans un répertoire nommé `META-INF` :

```
# unzip -l testjar.jar
Archive:  sources/jar/testjar.jar
  Length      Date    Time    Name
-----
      0  09-18-09  09:31    META-INF/
     75  09-18-09  09:31    META-INF/MANIFEST.MF
    884  09-18-09  09:19    Compte.class
    898  09-18-09  09:19    TestJAR.class
-----
   1857
                   4 files
```

Le démarrage d'une application stockée dans une archive JAR requiert l'utilisation d'une option spécifique. Pour lancer notre application `testjar`, nous utilisons donc la ligne de commande :

```
# java -jar testjar.jar
```

5. Correction des Exercices

Exercice, section 2.2.1

```
public class StringParser {

    public static void main (String[] args) {
        if (args.length == 0) {
            error();
        }

        // args[0] contient une chaine de type: Jean-Michel Caricand:16 route
de Gray:25030:Besançon
        String str = args[0];
        StringBuffer buffer = new StringBuffer();
        int pos1 = 0;
        int pos2 = 0;

        // on recupere le nom
        pos2 = str.indexOf(":", pos1);
        if (pos2 == -1) {
            error();
        }
        String nom = str.substring(pos1, pos2);
        // il faut trouver le nom pour le mettre en majuscule
        pos1 = nom.indexOf(" ");
        if (pos1 == -1) {
            error();
        }
        buffer.append(nom.substring(0, pos1) + " ");
        pos1++;
        buffer.append(nom.substring(pos1, pos2).toUpperCase() + "\n");

        // on recupere l'adresse
        pos1 = pos2 + 1;
        pos2 = str.indexOf(":", pos1);
        if (pos2 == -1) {
            error();
        }
        buffer.append(str.substring(pos1, pos2) + "\n");

        // on recupere le code postal
        pos1 = pos2 + 1;
```

```

        pos2 = str.indexOf(":", pos1);
        if (pos2 == -1) {
            error();
        }
        buffer.append(str.substring(pos1, pos2) + " ");

        // on recupere la ville
        pos1 = pos2 + 1;
        pos2 = str.length();
        if (pos2 == -1) {
            error();
        }
        buffer.append(str.substring(pos1, pos2).toUpperCase() + "\n");

        System.out.println(buffer.toString());
    }

    public static void error () {
        System.out.println("Il faut passer une chaine de caracteres, par
exemple: Jean-Michel Caricand:16 route de Gray:25030:Besançon");
        System.exit(-1);
    }
}

```

Exercice 1, section 2.7.4.1

```

public class Exercice1 {
    public static void main(String[] args) {
        // Somme des entiers impairs situés entre 1 et 99 en utilisant la
        // boucle for.
        int somme = sommeEntier();
        System.out.println("Somme des entiers impairs situés entre 1 et 99: " +
        somme);

        // Calcul de la valeur de 2.5 à la puissance 3 en utilisant la méthode
        // pow.
        double value = calculAvecPow();
        System.out.println("Calcul de la valeur de 2.5 à la puissance 3: " +
        value);

        // Imprime les entiers de 1 à 20 en utilisant une boucle while.
        // Utilisez la variable « i »
        // dans l'expression booléenne de l'instruction while. Imprimez
        // seulement cinq entiers par ligne .
        String str = imprimeEntier();
        System.out.println("Imprime les entiers de 1 à 20: " + str);
    }

    private static String imprimeEntier() {
        StringBuffer buffer = new StringBuffer();
        int i = 1;
        while (i <= 20) {
            buffer.append(i + "\t");
            if (i % 5 == 0) {
                buffer.append("\n");
            }
            i++;
        }
        return buffer.toString();
    }
}

```

```

    }

    private static double calculAvecPow() {
        return Math.pow(2.5, 3);
    }

    private static int sommeEntier() {
        int somme = 0;
        for (int i = 1; i < 100; i++) {
            if (i % 2 == 1) {
                somme += i;
            }
        }
        return somme;
    }
}

```

Exercice 2, section 2.7.4.2

```

i = 1; => il manque le type : int i = 1 ;

while ( i <= 10 ) ; => remplacer ; par {
    i++;
}

for ( k = 0.1; k <= 1.0 ; k += 0.1 ) => il manque le type : for (double k = ...
    System.out.println( k );

=> Il faut déclarer et initialiser n : int n = 1 ;
switch ( n )
{
    case 1:
        System.out.println( "Le nombre est 1" );
        => il manque un break
    case 2:
        System.out.println( "Le nombre est 2" );
        break;
    default:
        System.out.println( "Le nombre est différent de 1 et de 2" );
        break;
}

```

```

n = 1 ; => il manque le type : int n = 1 ;

while ( n <> 10 ) => <> n'est pas un opérateur java : <=
    System.out.println( n++ );

```

Exercice, section 2.7.4.3

```

counter = 2; => il manque le type : int counter = 2 ;
do {
    System.out.println( counter );
    counter += 2 ;
} While ( counter < 100 ); => while avec une minuscule
=> la condition de sortie est fausse, le nombre 100 ne sera pas imprimé

```

Exercice, section 2.7.4.4

```
public class Exercice {
    public static void main(String[] args) {
        byte value = 1;
        byte multiply = 2;
        System.out.println("value: " + value);
        for (int i = 0; i < 8; i++) {
            value = (byte) (value * multiply);
            System.out.println("value: " + value);
        }
    }
}
```

Résultat :

```
value: 2 => valeur en bit: 0000 0010
value: 4 => valeur en bit: 0000 0100
value: 8 => valeur en bit: 0000 1000
value: 16 => valeur en bit: 0001 0000
value: 32 => valeur en bit: 0010 0000
value: 64 => valeur en bit: 0100 0000
value: -128 => valeur en bit: 1000 0000 : comme le type byte est signé cette
valeur est donc négative
value: 0 => valeur en bit: 0000 0000 : En multipliant -128 * 2 nous obtenons
256. Ce nombre « int » est représenté sur 2 octet : 0000 0001 0000 0000.
Lorsqu'on converti en « byte » nous java conserve que le deuxième octet : 0000
0000
```

Exercice, section 2.7.4.5

```
public class Exercice {
    public static void main(String[] args) {
        Random r = new Random();

        for (int i = 0; i < 20; i++) {
            char lettre = (char) (64 + r.nextInt(26));
            if (lettre == 'A' || lettre == 'E' || lettre == 'I' ||
                lettre == 'O' || lettre == 'U' || lettre == 'Y')
            {
                continue; // ne pas imprimer de voyelle
            }
            System.out.println(lettre);
        }
    }
}
```

Exercice, section 2.7.4.6

```
public class Exercice {
    public static void main(String[] args) {
        System.out.print("Entrer le rayon: ");
        int radius = LectureClavier.lireInt();
        double volume = ( 4.0 / 3.0 ) * Math.PI * Math.pow( radius, 3 );
        System.out.println("Volume: " + volume);
    }
}
```

```
}  
}
```

Exercice, section 2.8.1.1

```
import java.util.Arrays;  
  
public class Exercice1 {  
    public static void main(String[] args) {  
        Exercice1 ex = new Exercice1();  
        int [] tableau = { 56, 45, 3, 67, 67, 45, 78, 26 };  
        System.out.println("Tableau initial: " + Arrays.toString(tableau));  
        while (ex.triBulles(tableau)) {  
            System.out.println("Tableau en cours: " +  
Arrays.toString(tableau));  
        }  
        System.out.println("Tableau trie: " + Arrays.toString(tableau));  
    }  
  
    public boolean triBulles (int[] tab) {  
        boolean aRefaire = false;  
        for (int i = 0; i < tab.length; i++) {  
            if (i < tab.length - 1) {  
                if (tab[i] > tab[i + 1]) {  
                    int tmp = tab[i];  
                    tab[i] = tab[i + 1];  
                    tab[i + 1] = tmp;  
                    aRefaire = true;  
                }  
            }  
        }  
        return aRefaire;  
    }  
}
```

Exercice, section 2.8.1.2

```
import java.text.DecimalFormat;  
import java.util.Random;  
  
public class Matrice {  
  
    public static void main(String[] args) {  
        // creation de la matrice  
        float[][] matrice = new float[10][10];  
  
        Random random = new Random();  
        for (int i = 0; i < 10; i++) {  
            for (int j = 0; j < 10; j++) {  
                matrice[i][j] = random.nextFloat();  
            }  
        }  
  
        // impression matrice  
        DecimalFormat format = new DecimalFormat("0.00");  
        for (int i = 0; i < 10; i++) {  
            for (int j = 0; j < 10; j++) {
```

```

        System.out.print (format.format(matrice[i][j]) + "
");
    }
    System.out.println();
}
}
}

```

Exercice, section 2.8.1.3

```

import java.util.Arrays;

public class Exercice1 {
    public static void main(String[] args) {
        Exercice1 ex = new Exercice1();
        String [] tableau = { "Python", "Basic", "Perl", "C++", "Java"};
        System.out.println("Tableau initial: " + Arrays.toString(tableau));
        while (ex.triBulles(tableau)) {
            System.out.println("Tableau en cours: " +
Arrays.toString(tableau));
        }
        System.out.println("Tableau trie: " + Arrays.toString(tableau));
    }

    public boolean triBulles (String[] tab) {
        boolean aRefaire = false;
        for (int i = 0; i < tab.length; i++) {
            if (i < tab.length - 1) {
                if (tab[i].compareToIgnoreCase(tab[i + 1]) > 0) {
                    String tmp = tab[i];
                    tab[i] = tab[i + 1];
                    tab[i + 1] = tmp;
                    aRefaire = true;
                }
            }
        }
        return aRefaire;
    }
}

```

Exercice, section 2.8.1.4

```

import java.util.Arrays;
import java.util.regex.Pattern;

public class Exercice1 {
    public static void main(String[] args) {
        Exercice1 ex = new Exercice1();
        Pattern p = Pattern.compile(" ");
        String [] tableau = p.split("Ecrivez un programme qui initialise une
variable de type String avec un texte de votre choix.");
        System.out.println("Tableau initial: " + Arrays.toString(tableau));
        while (ex.triBulles(tableau)) {

```

```

        System.out.println("Tableau en cours: " +
Arrays.toString(tableau));
    }
    System.out.println("Tableau trie: " + Arrays.toString(tableau));
}

public boolean triBulles (String[] tab) {
    boolean aRefaire = false;
    for (int i = 0; i < tab.length; i++) {
        if (i < tab.length - 1) {
            if (tab[i].compareToIgnoreCase(tab[i + 1]) > 0) {
                String tmp = tab[i];
                tab[i] = tab[i + 1];
                tab[i + 1] = tmp;
                aRefaire = true;
            }
        }
    }
    return aRefaire;
}
}

```